Enterprise Service Intake

Architecture & Design Brief - V2

Candidate	Forrest Zhang
Role	Power Platform Senior Developer - Take-Home Case
Prepared For	Mitacs hiring and technical review team
Environment	https://mitacs.crm.dynamics.com/
Portal	https://enterprise-service-intake-hellox.powerappsportals.com
Status	Candidate submission brief - V2

Executive Summary

This candidate submission implements an enterprise-grade external service request intake process using Power Pages, Dataverse, Power Automate, C# plugins, and PCF. External customers submit authenticated multi-step requests, receive real-time routing and SLA feedback, and upload supporting documentation to SharePoint after submission. Dataverse holds the system of record while server-side plugins apply non-bypassable routing and close-validation rules. Power Automate handles applicant confirmation email, manager approval, mock ERP synchronization, and resilient error logging.

End-to-End Outcome

1. Customer submits a portal request and receives a formatted confirmation number.
2. Customer can upload supporting files to the SharePoint document library associated to the saved Service Request.
3. Dataverse plugin applies the matching routing/SLA rule and flags approval or documentation requirements.
4. Internal coordinator reviews the request in the model-driven app with a PCF status indicator.
5. Critical or high-priority items route to manager approval.
6. Power Automate sends the confirmation number to the applicant and records failures in System Error Logs.
7. Approved requests are posted to a mock REST endpoint and the external system ID is written back to Dataverse.
8. Failures in approval or integration are captured in a custom System Error Log table.

Live Review Access

These links and accounts are included so the hiring team can validate the submitted solution end to end during the review.

Area	Value	What Reviewers Can Validate
Environment	https://mitacs.crm.dynamics.com/	Open Dataverse tables, model-driven app, flow runs, and solution components.
Maker Solution	https://make.powerapps.com/environments/99dd50ed-a753-e37f-912c-78a022b12b09/solutions	Review solution-aware components and exports.
Model-driven App	https://mitacs.crm.dynamics.com/main.aspx?appid=3de4f813-b454-f111-bec7-000d3a3aca8f	Review request queue, form fields, and PCF coordinator experience.
Power Pages Site	https://enterprise-service-intake-hellox.powerappsportals.com	Submit a customer request, verify dynamic SLA/routing preview, and open post-submit SharePoint upload.
Cloud Flows	ESI - Send Confirmation Email; ESI - Approval and ERP Sync	Review applicant email, approval, HTTP sync, run history, and Try/Catch error handling.

Live Review Accounts

Account	Purpose
forrest@hellosmart.ca	Admin/customizer reviewer
agent@hellosmart.ca	Internal service coordinator
manager@hellosmart.ca	Approval manager

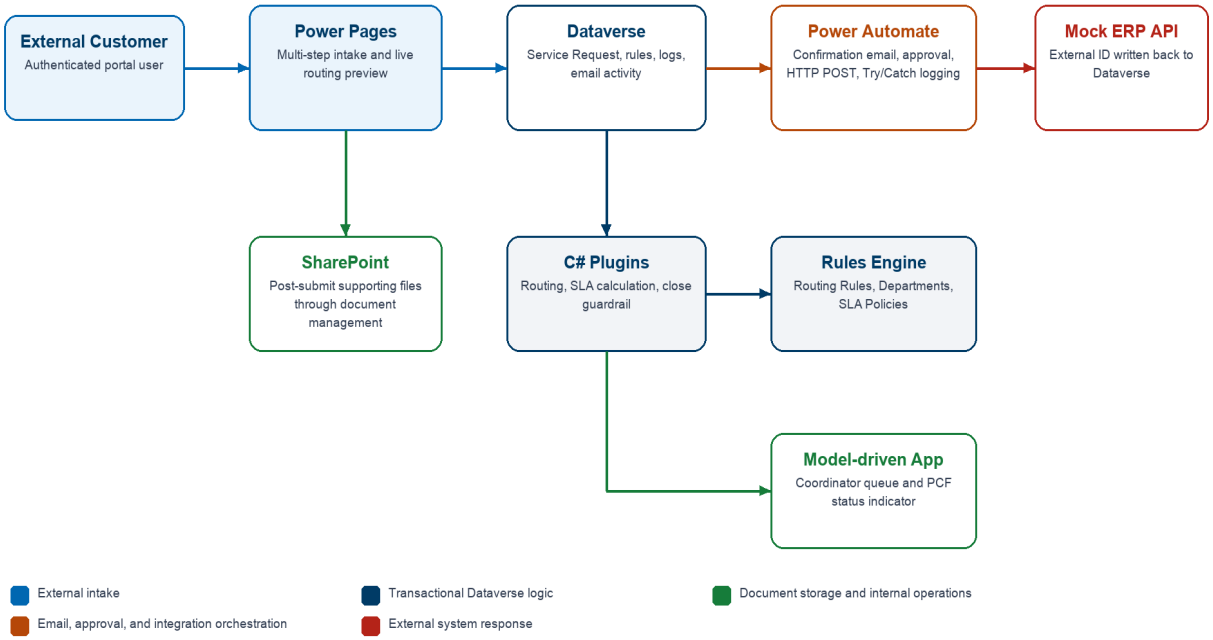
Architecture Overview

The solution keeps Dataverse as the authority for state while placing business logic in the component that best enforces the requirement.

Enterprise Service Intake - Runtime Architecture

Customer intake, Dataverse rules, SharePoint documents, confirmation email, approval, and mock ERP sync

Arrowheads show runtime flow direction; colors indicate the owning platform area.



Requirement	Component	Reason
External intake	Power Pages	• Authenticated customer access • Contact-scoped permissions • Multi-step forms • Post-submit SharePoint upload • Liquid/Web API extensibility
System of record	Dataverse	• Relational model • Security roles and ownership • Auditing and solution packaging • Consistent state across portal, app, flow, and APIs
Routing and SLA	C# plugin	• Runs transactionally on create/update • Same rule result for all entry points • Cannot be bypassed by imports, flows, APIs, or alternate clients
Close guardrail	C# plugin	• Enforces critical documentation requirements • Runs at the server boundary
Applicant email, approval, and ERP sync	Power Automate	• Sends the confirmation email • Handles human approval • Calls the mock ERP API • Provides run history, retry visibility, and Try/Catch logging
Internal coordinator UX	Model-driven app + PCF	• Secure operational CRUD • Coordinator queue views • Compact visual severity, SLA, approval, and sync status

Dataverse Data Strategy

The data model separates request state, rule configuration, documents, integration telemetry, and operational error records.

Dataverse Entity Relationship Diagram

Core tables support request intake, routing, SLA assignment, SharePoint documents, integration logs, and error handling.

Arrowheads point to the dependent/many-side table; labels show relationship cardinality.

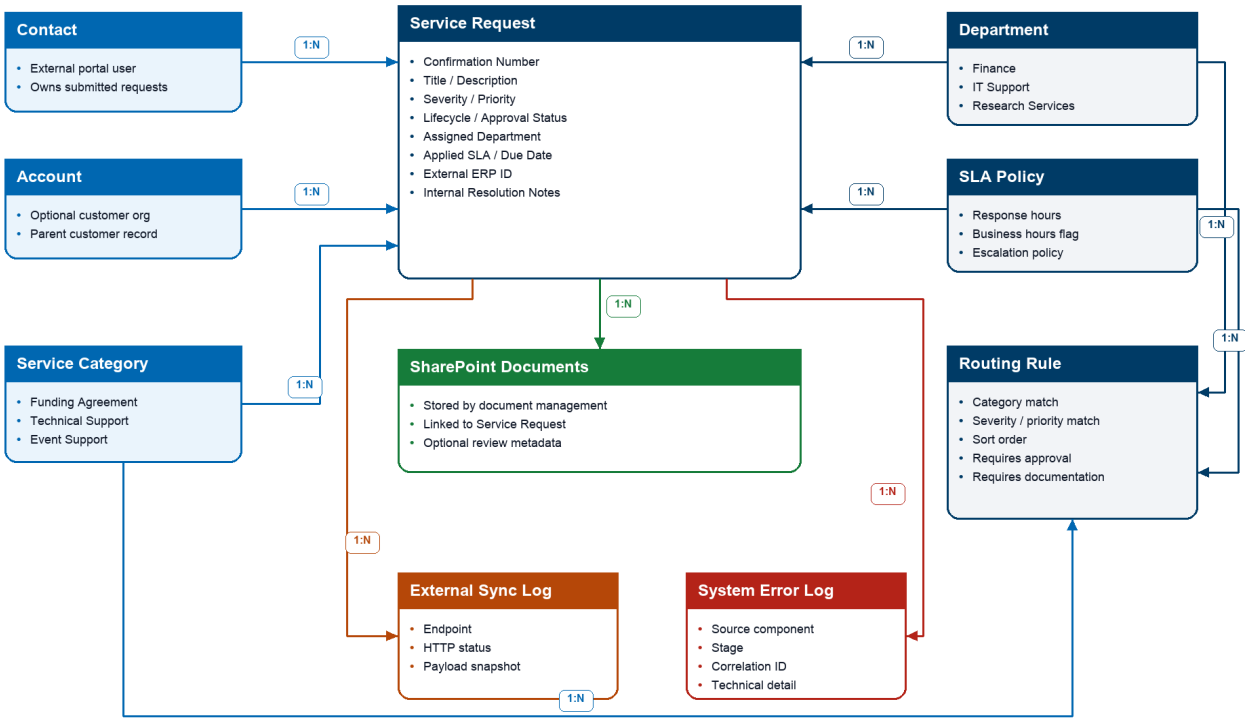


Table	Purpose	Key Relationships
Service Request	• Primary intake and work-management row • Stores confirmation number, status, priority, SLA, approval state, external ERP ID, and internal notes	• Contact • Account • Service Category • Department • SLA Policy • SharePoint Documents • External Sync Log • System Error Log
Routing Rule	• Configurable rules engine row • Matches category, severity, and priority • Supplies department, SLA, approval, and documentation requirements	• Service Category • Department • SLA Policy
Department	• Internal routing destination • Operational ownership for assigned requests	• Service Request • Routing Rule
SLA Policy	• Response target • Business-hours and escalation policy assigned by routing	• Service Request • Routing Rule
SharePoint Documents	• Supporting files stored through Power Pages document management • Files linked to the saved Service Request after creation	• Service Request • SharePoint Document Location
External Sync Log	• Integration audit trail • Tracks outbound ERP endpoint, status, and payload snapshot	• Service Request

Table	Purpose	Key Relationships
System Error Log	• Structured error capture • Used by plugin, flow, and integration failure paths	• Service Request optional

Confirmation Number

- Service Request uses a Dataverse autonumber format: SR-{yyyyMMdd}-{SEQNUM}.
- The number is generated server-side, remains stable after creation, and is safe to show externally.
- The formatted value gives customers a readable reference while preserving Dataverse row IDs for integration and support work.

Security Strategy

The design assumes least privilege across external portal users, internal coordinators, managers, and administrators.

Power Pages Access

- External users authenticate to Power Pages and are associated to Contact rows.
- Table permissions are contact-scoped so users can create requests and view only requests tied to their contact.
- Reference data needed for dynamic preview is exposed read-only.
- Portal forms exclude internal-only fields, integration payloads, internal notes, sync logs, and system error logs.

Internal Dataverse Access

- Coordinators use the model-driven app for operational request triage.
- Managers review approvals and can inspect high-priority items.
- Sensitive internal resolution notes are kept off the portal and should be protected with role and column-level security in production.
- Error and sync telemetry tables are intended for managers, admins, and support owners rather than broad staff access.

Security Tradeoffs

Area	Lean Demo Choice	Production Hardening
Portal identity	• Authenticated Power Pages users in the interview tenant	• Use Entra External ID or configured B2C provider • Apply lifecycle controls • Add conditional access where appropriate
Uploads	• Post-submit document-management page • Files stored in SharePoint for the saved Service Request	• Apply file type limits • Use malware scanning • Review retention and DLP rules
Secrets	• No tenant passwords or tokens committed to Git	• Use connection references and environment variables • Use Key Vault-backed custom connectors • Prefer managed identities where available

External User Experience

The portal is scoped as a clean customer intake experience rather than an internal operations surface.

Submission Flow

1. User signs in to the Power Pages site.
2. User starts a new service request and enters request details.
3. User selects category, severity, and priority.
4. Portal dynamically previews expected department, SLA target, approval requirement, and documentation requirement before final submission.
5. User reviews and submits the request, then receives a confirmation number.
6. User opens the secure document upload step and adds files to the SharePoint folder associated with the saved request.

Dynamic Preview Implementation

- The portal reads eligible routing/SLA data through Power Pages Web API and/or Liquid-rendered data.
- Client-side JavaScript recalculates the displayed routing destination and expected SLA when the user changes category, severity, or priority.
- The preview avoids a full page reload and mirrors the server-side plugin rule result, while the plugin remains the authoritative enforcement layer.

Automation And Integration

The cloud flow handles long-running human approval and integration work that should remain visible in run history.

Stage	Implementation
Trigger	• Dataverse row added or modified • Approval required • Approval pending • Integration unsynced
Try scope	• Start and wait for manager approval • Branch on approval decision • Call mock ERP API • Update Service Request • Write External Sync Log
HTTP sync	• POST approved request details to https://hellox.ca/api/esi-service-requests • Store returned external ID in Dataverse
Reject branch	• Mark approval rejected • Skip ERP sync
Catch scope	• Runs after failure, timeout, or skipped Try scope • Writes System Error Log with flow run correlation details • Marks request failed where appropriate

Flow: ESI - Send Confirmation Email

Stage	Implementation
Trigger	• Dataverse row added for Service Request
Try scope	• Validate applicant Contact and email • Send HTML confirmation through Office 365 Outlook Send an email (V2) • Include confirmation number, request title, and submitted timestamp • Update customer-visible notes

Stage	Implementation
Skip path	• Missing Contact or email writes a System Error Log • Flow avoids silent failures
Catch scope	• Runs after failure, timeout, or skipped Try scope • Writes System Error Log with flow run correlation details

Resiliency Pattern

- Try/Catch scopes make failures observable and reviewable during the live demo.
- The custom System Error Log captures source component, stage, message, technical detail, correlation ID, and related request.
- Request integration status is updated to failed when the integration path cannot complete.
- Office 365 Outlook action output, Flow run history, and Approval records provide proof if tenant email delivery is restricted.

Pro-Code Extensibility

The pro-code pieces are intentionally placed where low-code would either be bypassable or less effective.

Component	Location	Responsibility	Why Here
ServiceRequestRoutingPlugin	src/plugins/ServiceIntake.Plugins/ServiceRequestRoutingPlugin.cs	• Evaluates routing rules • Assigns department and SLA • Sets due date and lifecycle defaults • Sets approval and documentation requirements	• PreOperation server-side execution • Transactional rule result • Consistent across portal, model app, imports, flows, and APIs
ServiceRequestClosureGuardPlugin	src/plugins/ServiceIntake.Plugins/ServiceRequestClosureGuardPlugin.cs	• Blocks critical request closure • Requires internal resolution notes • Requires documentation evidence	• Server-side validation • Prevents bypass from forms, imports, flows, and APIs
SlaStatusIndicator PCF	src/pcf/SlaStatusIndicator/	• Displays severity • Shows SLA, approval, and sync status • Keeps coordinator status scanning compact	• Improves internal model-driven usability • Does not duplicate authoritative business logic
Provisioning Utility	src/scripts/ServiceIntake.Provisioning/	• Creates metadata • Registers plugins • Provisions app, forms, views, dashboards, and sample data • Patches flow definitions	• Repeatable environment build • Reviewable ALM steps

ALM, Verification, And Demo Evidence

The repository contains both deployable solution packages and unpacked source so reviewers can inspect the implementation.

Deliverables

Artifact	Location
Managed solution	solution/export/Enterprise_ServiceIntake_ForrestZhang_managed.zip
Unmanaged solution	solution/export/Enterprise_ServiceIntake_ForrestZhang_unmanaged.zip
Unpacked solution source	solution/unpacked/managed/ and solution/unpacked/unmanaged/
C# plugin source	src/plugins/ServiceIntake.Plugins/
PCF source	src/pcf/SlaStatusIndicator/
Power Pages source	src/powerpages/ and powerpages-live/
Demo script	docs/demo/demo-script.md
Verification notes	docs/evidence/verification.md

Verified Behavior

Check	Evidence
Plugin build	Passed.
Provisioning utility	• Builds with zero warnings • NuGet vulnerability audit reports no vulnerable packages
PCF	Build and pac pcf push passed.
Power Pages	• Dynamic preview shows Finance • 4 hour SLA and approval required display before submit • Post-submit SharePoint upload is available from the confirmation dialog
Portal submission	• Portal demo request created with formatted confirmation SR-20260521-001004 • Outlook confirmation smoke test created SR-20260521-001018
Closure guard	• Blocked undocumented critical closure • Allowed documented closure
Flows	• Approval/ERP flow is active and solution-aware • Confirmation email flow is active and solution-aware • Both use Try/Catch logging patterns
Solution export/unpack	Managed and unmanaged exports and unpacked source generated successfully.

Known Demo Notes

- If email delivery is restricted, use Office 365 Outlook action output, Approval records, and Flow run history as evidence.
- The HelloX mock ERP endpoint avoids third-party API keys and includes a deliberate failure mode for Catch-scope demos.
- The hidden HelloX console at https://hellox.ca/esi/ is noindex and not linked from public navigation.
- If a target environment does not automatically bind the PCF control on import, add the PCF control to the coordinator form field in the form designer.

Live Demonstration Path

The demo is designed to show the full lifecycle and then drill into implementation decisions.

1. Walk through the ERD and architecture choices.
2. Submit a critical funding request from the portal and show dynamic preview before submit.
3. Open the request in the model-driven app and review routing, SLA, approval, sync status, dashboards, and internal fields.
4. Open the cloud flow and explain approval, HTTP POST, writeback, sync log, reject branch, and Catch scope.
5. Demonstrate or explain failure handling through System Error Log rows.
6. Demonstrate the critical close guardrail through the smoke test or form behavior.
7. Show managed solution, unpacked source, PCF, plugin source, Power Pages source, and GitHub repository.

Prepared Q&A

Question	Answer
Why plugin over flow for routing?	Routing must be transactional and consistent for every entry point, including portal, model app, imports, flows, and APIs.
Why plugin for close validation?	The requirement says agents cannot bypass documentation requirements; server-side PreOperation validation is the correct enforcement point.
Why flow for approvals and ERP sync?	Approvals and integration orchestration need human workflow, connector history, retry visibility, and run evidence.
How do customers only see their data?	Power Pages users are tied to Contacts, and contact-scoped table permissions restrict request visibility to owned rows.
How is this deployable?	Components are solution-aware, exported as managed/unmanaged zips, unpacked with PAC, and source-controlled with raw plugin and PCF code.